

CommunityGuard-AI: Agentic AI for Cyber-Physical Defense of Energy Communities

Maanat Bhadani, Alaa Selim, and Junbo Zhao

Thayer School of Engineering, Dartmouth College

Hanover, NH, USA

{maanat.h.bhadani.29, alaa.e.selim, junbo.zhao}@dartmouth.edu

Abstract—Energy communities rely on accurate measurements to coordinate buildings, solar generation, and batteries. Detecting corrupted data is only the first step: a poor repair can introduce further errors. This paper presents CommunityGuard-AI, a four-stage framework that connects numerical evidence, incident diagnosis, repair planning, and verification. Its contribution is a complete response workflow that selects a repair tool, checks individual changes against peer references and a protected energy balance, and retains received data when no proposal passes verification. LangGraph manages execution, bounded retries, and decision records. A conditional proof states when an accepted change reduces measurement error. We evaluated the framework with rule-based decisions on two CityLearn building groups comprising 1920 attacked records and 32 clean controls. It improved 1613 attacked records, left 296 unchanged, and worsened 11, reducing mean normalized error by 79.2%. No clean control was harmed. A supplementary comparison examined Llama as the diagnosis and planning component. Both Llama configurations improved 40 of 48 attacked records, compared with 41 for rules. Neither Llama configuration harmed a subset record. Actual responses, token counts, and processing times documented the language component’s behavior and workload. The study demonstrated selective, traceable offline telemetry repair under explicit sensor-trust and reference-accuracy assumptions.

Index Terms—energy communities, telemetry repair, verification, agentic AI, cyber-physical security

I. INTRODUCTION

Energy communities coordinate building demand, photovoltaic (PV) generation, battery energy storage systems (BESSs), and price-responsive devices through shared measurements. Their operation depends on knowing how much energy buildings consume, how much renewable generation is available, and when that energy is needed. Corrupted load, PV, price, or timing information can mislead the decisions that coordinate these resources, linking telemetry integrity to the cyber-physical security of distribution systems [1], [2]. Smart-inverter interfaces and synchronization signals provide concrete examples of this exposure [3], [4].

Detecting an unusual value does not explain how to repair it. A false-data-injection attack can preserve realistic magnitudes and daily patterns [5]. For example, an under-reported load can still follow its normal daily shape. The defender must identify the affected building and measurement channel, choose a reliable replacement, and avoid changing healthy data. A poor repair can add errors to the information supplied to a controller. The challenge is to connect detection to a limited, testable response.

Anomaly detectors address part of this challenge. Autoencoders learn normal patterns and identify deviations without labeled examples of every attack [6]. However, their reconstructed values are estimates: a replacement can fit the model better while moving farther from the true measurement. Energy balance and measurements from other buildings provide additional checks. These checks depend on trusted sensors and accurate reference estimates. A repair method must state these assumptions, limit which values it changes, and keep the received data when the evidence is insufficient.

A complete response workflow must connect evidence to action while keeping proposed repairs subject to numerical checks. Large language models (LLMs) can contribute incident interpretation and tool selection [7], [8], although their diagnoses and explanations can be unsupported [9]. Digital-twin and constrained-LLM studies have combined numerical evidence with language reasoning [10], [11]. For telemetry defense, the central design question is how to organize diagnosis, repair tools, verification, and fallback behavior into a response whose effect can be measured independently.

CommunityGuard-AI addresses this question through four stages: numerical evidence, incident diagnosis, repair planning, and verification. The diagnosis and planning stages support rule-based or Llama decisions. Both use the same repair tools and numerical guard. LangGraph manages evidence requests, one allowed retry, accepted changes, and abstention (keeping the received data). Llama contributes language-based interpretation and tool selection within this structure. The framework’s response conditions apply to either decision policy.

The paper makes three contributions: (i) an integrated response workflow that connects building/channel diagnosis to executable repair tools, selective changes, and bounded fallback; (ii) a verification rule using calibrated references and a protected energy balance, with a conditional proof of error reduction; and (iii) an evaluation on two building groups using later data for testing and withheld clean measurements for scoring. The evaluation measures recovery, errors introduced into healthy data, and failure cases. A supplementary Llama comparison examines language-based decisions, execution traces, token use, and latency within the same framework.

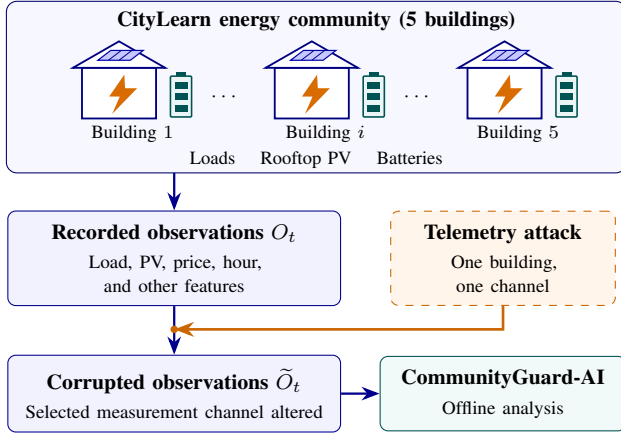


Fig. 1. Observation flow in the energy-community case study. Building models generate measurements under zero control actions. Each attack alters one reported channel in one building before offline analysis. Arrows show data flow.

II. SYSTEM AND THREAT MODEL

A. Energy-Community Model

We consider N heterogeneous CityLearn buildings, indexed by $\mathcal{B} = \{1, \dots, N\}$, with non-shiftable loads, PV generation, and electrical storage. Figure 1 shows their observation flow.

At hourly step t , the observation $O_t \in \mathbb{R}^{N \times M}$ contains M features per building. All features enter the baseline observation vector; the attacked channels form the subvector

$$s_{i,t} = [P_{i,t}^{\text{load}}, P_{i,t}^{\text{PV}}, C_{i,t}, H_{i,t}]^T, \quad (1)$$

where the entries denote non-shiftable load, PV generation, electricity price, and the reported hour, respectively.

Every CityLearn action is zero during collection. Corrupted observations are processed offline and do not drive charging or discharging; the experiment measures telemetry repair rather than changes in delivered power. Building labels are one-based.

B. General Adversarial Model

An adversary modifies selected entries of the returned observations. The corrupted observation is represented by

$$\tilde{O}_t = O_t + \delta_t, \quad (2)$$

where $\delta_t \in \mathbb{R}^{N \times M}$ is nonzero on attacked entries. Each experiment corrupts one channel in one building throughout a 168-hour record. The objective is localization and repair; network intrusion is not simulated.

1) *Market Hack: False-Data Injection on Price Telemetry:* The attacker scales the electricity-price signal and adds stochastic noise:

$$\tilde{C}_{i,t} = \lambda C_{i,t} + \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, \sigma^2), \quad (3)$$

where λ controls the distortion and σ is the noise standard deviation.

2) *Meter Hack: Proportional Load Manipulation:* The attacker modifies the reported non-shiftable load according to

$$\tilde{P}_{i,t}^{\text{load}} = \alpha P_{i,t}^{\text{load}}, \quad (4)$$

where α changes the reported demand while retaining its temporal pattern.

3) *Inverter Hack: PV-Sensor Blinding:* The attacker multiplies the reported PV output by a bounded random factor:

$$\tilde{P}_{i,t}^{\text{PV}} = P_{i,t}^{\text{PV}} u_t, \quad u_t \sim \mathcal{U}(\beta_{\min}, \beta_{\max}). \quad (5)$$

The perturbation affects the reported PV output, not the inverter control loop.

4) *Time Spoofing: Hour-Feature Offset:* The implemented timing perturbation is

$$\tilde{H}_{i,t} = ((H_{i,t} - 1 - \Delta h) \bmod 24) + 1, \quad (6)$$

where Δh offsets the reported hour within CityLearn's valid domain 1–24; simulator time advances normally.

C. Reference Availability and Trust Boundary

Ordinary records contain one compromised building/channel and an uncompromised peer majority. The separate net-electricity-consumption observation is protected, and the zero-action trajectory obeys

$$n_{i,t} = \ell_{i,t} - p_{i,t}, \quad (7)$$

where n , ℓ , and p denote net import, load, and PV energy per step. The collected development data satisfy this identity to floating-point precision. Active storage, thermal demand, or other devices would require independently measured additional terms.

The defender assumes that the net meter and the other buildings are trustworthy; it does not receive the clean values of the attacked channel. A separate test corrupts load and net import together to examine this assumption. Clean test values are used only to score the completed response.

III. COMMUNITYGUARD-AI FRAMEWORK

Figure 2 shows CommunityGuard-AI's four stages. Numerical evidence identifies inconsistencies; the decision policy diagnoses a building/channel and selects a repair tool; the guard checks each proposed change. LangGraph manages optional evidence requests, acceptance, retry, and abstention. Rules and Llama are two implementations of the diagnosis and planning stages; references, repair tools, and verification are shared.

A. Node 1: Feature-Aware Numerical Evidence

The feature-aware autoencoder uses price, load, and PV scaled using training data, together with $\sin(2\pi H/24)$ and $\cos(2\pi H/24)$. These two clock features keep adjacent hours close across midnight. The input dimension is $5N$, and layer widths are $5N-64-32-64-5N$, with ReLU hidden activations. Month is excluded from this reconstruction. Training uses 300 full-batch Adam steps, learning rate 0.01, and seed 42.

Separate reference estimates support the checks. Price and hour use the median of other buildings' measurements. For

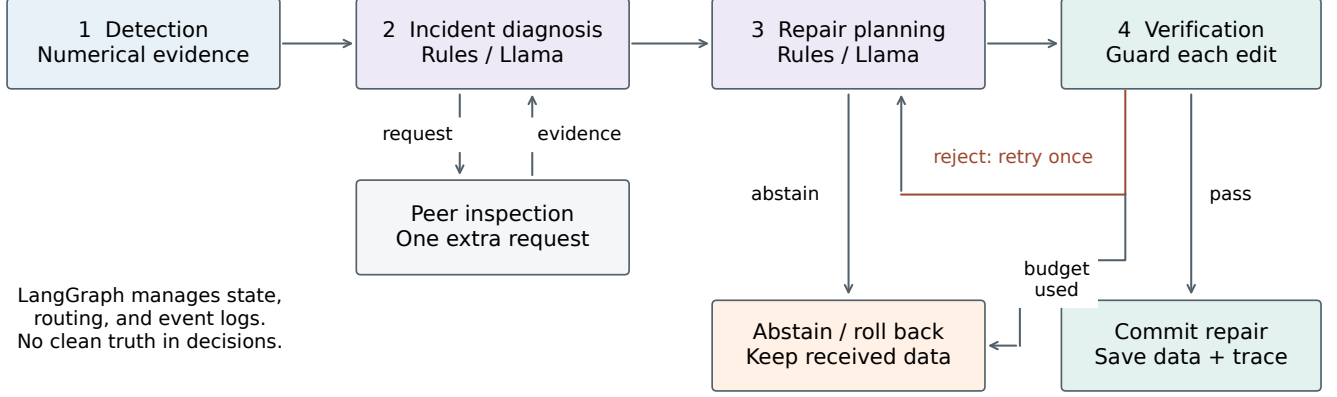


Fig. 2. CommunityGuard-AI response workflow. The diagnosis and planning stages use rules or Llama; both share the numerical evidence, tools, and guard. LangGraph records decisions and routes accepted changes, optional evidence requests, one allowed retry, and abstention. The supplementary comparison examines the Llama implementation within this structure.

target PV p_i , each other building j supplies a nonnegative scalar transfer $a_{ij} = \max\{0, \sum_t p_{j,t} p_{i,t} / \sum_t p_{j,t}^2\}$ fitted on training data. The three peers with the smallest training prediction error are retained. The reference is the median of their transferred values,

$$q_{i,PV,t} = \text{median}_{j \in \mathcal{P}_i} a_{ij} p_{j,t}, \quad |\mathcal{P}_i| = 3. \quad (8)$$

Peers with zero PV energy are excluded. Scaling allows different PV capacities, but useful relationships between buildings are still required. The load reference is $q_{i,\text{load},t} = \max\{0, n_{i,t} + q_{i,PV,t}\}$. It does not use the target building's received PV value, which helps distinguish a load attack from a PV attack.

For reference $q_{i,k,t}$ and received measurement $x_{i,k,t}$, calibration supplies

$$\delta_{i,k} = \max\{Q_{0.90}(|x - q|), 0.01 s_{i,k}\}, \quad (9)$$

where s is the training magnitude scale and $Q_{0.90}$ is the 90th percentile of the absolute reference error. Clock distance is measured around a 24-hour cycle, with a minimum radius of 0.01 hours. Calibration uses clean data separate from training and testing. The radius describes past reference errors; it does not guarantee accuracy in a new season.

A channel is reviewed when at least 10% of its 168 hourly samples are more than 2δ from the reference. For load/PV, those samples must also violate the protected energy balance by more than 10^{-4} . Samples that satisfy the balance receive zero excess score. This avoids flagging a healthy energy channel solely because a peer estimate is poor. Candidates are ranked by mean positive excess $\max\{|x - q|/(2\delta) - 1, 0\}$. The decision stage receives the top five building/channel candidates, their inconsistency fractions, reference means, radii, and trust assumptions. A large residual alone does not establish an attack.

B. Node 2: Incident Diagnosis and Evidence Request

The diagnosis stage receives numerical evidence without attack labels or clean test values. It returns a building, channel, assessment, optional evidence request, and short explanation. The rule-based policy selects the highest-scoring eligible candidate. The Llama policy interprets the supplied candidates and returns a structured response. Both can retain the input when no candidate is supported. The selected building/channel must appear in the supplied list; inconsistency alone does not prove malicious intent.

Llama can request one additional summary of peer measurements. LangGraph runs the tool, adds its results to the shared state, and returns to the forensic agent. The model can then confirm or revise its diagnosis, or abstain.

The evaluated language component uses locally served Llama 3.2 3B (Q4_K_M) through Ollama, with temperature zero, seed 42, 4096 context tokens, and at most 180 generated tokens per call [12], [13]. A JSON schema and a second strict parser enforce required fields and allowed labels. Malformed output causes abstention. Prompts and received outputs, server-reported token counts, model metadata, and call latency are saved separately from rule-based decision logs.

C. Node 3: Repair Planning and Executable Tools

The planning stage receives the diagnosis, reports for available tools, and verification feedback. Its decision policy chooses one of four operations or abstains:

- **PEER_CONSENSUS**: use the normalized peer estimate for the selected channel.
- **CONDITIONAL_MODEL**: use a ridge predictor trained on other buildings' corresponding channel and peer weather/calendar context; clock uses peer consensus.
- **AE_RECONSTRUCTION**: use the selected channel from the feature-aware autoencoder.

- **ENERGY_BALANCE**: reconstruct load from protected net import plus received PV, or PV from received load minus protected net import. This tool is available only for load/PV.

The ridge predictor uses standardized inputs and regularization coefficient 10. It excludes target-channel forecasts and clean test targets. Calendar inputs encode hour and month cyclically and day type categorically. Predicted physical quantities are clipped to nonnegative values, and hours are mapped to integers from 1 to 24.

The tools can produce different estimates, so tool selection affects the repair. Rules choose the eligible operation with the largest summed conditional improvement bound. Llama selects an operation from the same tool reports and supplies a short explanation. The explanation is saved for inspection; the selected operation and numerical guard determine the applied changes. This shared interface allows the decision policies to be examined within one response design.

D. Node 4: Numerical Checks and Selective Repair

For selected received value x , proposed value c , reference q , and radius δ , an entry is eligible for change only if

$$d(x, q) > d(c, q) + 2\delta, \quad (10)$$

where d is absolute distance, or circular distance for hours. The candidate must be closer to the reference by more than twice the uncertainty radius; implementation adds a 10^{-9} numerical margin. Load/PV candidates must also satisfy the protected energy balance within 10^{-4} native units. The guard checks finite values, nonnegative load/PV, and valid hours. It can accept a repair at some hours and reject it at others.

Let S contain only the accepted hours of the selected building/channel. The output is

$$y_j = \begin{cases} c_j, & j \in S, \\ x_j, & j \notin S. \end{cases} \quad (11)$$

All other channels keep their received values. A wrong diagnosis can still select a healthy channel, so limited editing alone does not guarantee recovery. Appendix A shows when the guard reduces error: the reference must be within δ of the true value.

E. LangGraph State, Replanning, and Rollback

The shared state stores received measurements, evidence, diagnosis, proposed operation, verification feedback, earlier attempts, and event history. It excludes clean truth and attack labels. Case identifiers appear only in logs. Proposed and accepted measurement arrays are stored separately.

LangGraph routes each case through abstention, optional peer inspection, verification, acceptance, or retry. It records rejected operations and prevents their repetition. At most two repair proposals and one extra evidence request are allowed. If parsing fails or no proposal is accepted within this budget, the original input is retained. In-memory checkpoints support execution; saved event logs and response arrays document each case [14].

The event trace links diagnosis, tool selection, numerical acceptance, and the final response.

IV. EXPERIMENTAL EVALUATION

A. Data, Development, and Chronological Evaluation

CityLearn 2.0.0 provides the Challenge 2022 cohorts [15]. All 8064 hours of the seven-building `phase_3` trajectory are designated as development data. Final evaluation uses `phase_1` and `phase_2`, each containing five different buildings with 28 observations each. The cohorts share the challenge environment and do not represent independently sampled climates.

For each final cohort, hours 0–4031 train the models, 4032–5375 calibrate the radii, and 5376–8063 provide 16 consecutive test weeks. The method, prompts, attack settings, subset selection, and illustration cases were fixed before examining these test results. Each cohort has its own training and calibration data; the method is fixed, but model weights are fitted separately.

Development compared calibration quantiles 0.80, 0.90, 0.95, and 0.99. The 0.90 setting was selected by maximum development recovery among settings that worsened no development records. A smaller radius can allow more repairs but also more mistakes. This choice did not guarantee zero harm on the test cohorts.

Three attack settings are evaluated at every building and week. The original setting uses price factor 0.6 with Gaussian noise standard deviation 0.02, load factor 0.7, PV factors in $[0.1, 0.5]$, and a four-hour clock offset. The mild setting uses 0.85 with noise 0.01, 0.85, $[0.6, 0.9]$, and one hour. An over-reporting setting uses 1.3 with noise 0.02, 1.3, $[1.3, 1.7]$, and an opposite four-hour offset. Thus recovery cannot rely solely on the direction of an under-reporting attack. The design yields $2 \times 16 \times 5 \times 4 \times 3 = 1920$ attacked records and 32 clean controls. Attack repetitions share operating windows and are not independent trajectories.

B. Framework Configurations and Recovery Metrics

The primary evaluation compares CommunityGuard-AI with rule-based decisions against no intervention, full-building autoencoder replacement, and single-channel feature-aware autoencoder replacement without the guard. Both autoencoders use the same cohort-specific training interval. The CommunityGuard-AI configuration selects the highest-scoring eligible building/channel and the operation with the largest summed positive conditional improvement bound. It exercises the framework's shared references, tool bank, selective verification, and LangGraph execution across the full benchmark.

For clean reference X^* , response Y , and training scales s_j , the reported error is

$$E(Y) = \frac{1}{TD} \sum_{t,j} \left(\frac{d_j(Y_{t,j}, X_{t,j}^*)}{s_j} \right)^2. \quad (12)$$

Here $T = 168$ hours and $D = 140$ measurement coordinates. Distance d_j is circular for hour and absolute otherwise. Scales

TABLE I
TELEMETRY RECOVERY ON ALL 1920 ATTACKED RECORDS

Method	Better	Same	Worse	Mean E
No intervention	0	1920	0	0.000285
Full-building AE	0	0	1920	0.026583
Feature-aware AE	831	0	1089	0.000167
CommunityGuard (rules)	1613	296	11	0.000059

Better/same/worse compare repaired and received-input errors. Lower E is better. CommunityGuard uses rule-based diagnosis and planning here; the Llama comparison uses a separate paired subset.

use the training 95th percentile with a 0.01 floor; hour, month, and day type use fixed scales of 24, 12, and 8. “Better,” “same,” and “worse” compare repaired and received-input errors with tolerance 10^{-12} . We also measure root-mean-square error (RMSE) on the attacked channel, error introduced into initially unchanged entries, detection, and correct building/channel selection. These metrics describe measurement accuracy.

C. Framework Recovery and Failure Analysis

CommunityGuard-AI with rules improves 1613 of 1920 attacked records, leaves 296 unchanged, and worsens 11. Mean normalized error falls from 0.000285 to 0.000059, a 79.2% reduction. Both building and channel are identified correctly in 1631 records; abstention does not count as a correct diagnosis. Full-building replacement worsens 1920 records, and single-channel feature-aware replacement worsens 1089. These comparisons evaluate the complete response designs; they do not isolate the guard from tool choice. Partial repairs and harmful outcomes remain in all reported totals.

Of 480 attacked records per channel, improved/unchanged/harmed counts are price 480/0/0; load 265/210/5; pv 388/86/6; clock 480/0/0. Mean per-record target RMSE decreases from 0.309 to 0.170 kWh for load and from 0.760 to 0.305 kWh for PV. These native-unit errors are averaged over all records of the corresponding attack, including abstentions. CommunityGuard-AI with rules raises 0 alarms and harms 0 of 32 clean controls.

Across 640 records per setting, improved/unchanged/harmed counts are mild: 481/156/3; strong under-reporting: 560/75/5; over-reporting: 572/65/3. The higher unchanged count for mild attacks indicates reduced intervention at lower perturbation magnitudes. The two cohorts yield 842/960 and 771/960 improved records, with 4 and 7 harmed records, respectively. All 11 harmful outputs select the correct building but confuse load and PV, editing the initially healthy complementary channel. This isolates channel attribution, rather than building selection, as the observed harmful failure mechanism.

D. Supplementary Decision-Policy Comparison

A supplementary comparison examines CommunityGuard-AI with Llama decisions on a subset fixed before evaluation: test weeks 1 and 9, four attack classes, three settings, and one rotating building per cohort/week/setting. The zero-based building index is $(1 + w + c + s) \bmod 5$, where w indexes the two selected weeks, c the two cohorts, and s the three settings.

This yields 48 attacked records and 4 clean controls. The same records are evaluated with rules; Llama is not used for all 1920 attacks.

Both language configurations use recorded Llama 3.2 3B responses obtained with the same prompts, schema, tools, and guard. The numerical repairs are recomputed; recorded responses are reused only when the new prompt matches the original exactly. One-pass Llama allows one diagnosis and one repair proposal. Full-graph Llama additionally permits a peer inspection and a second proposal after rejection. The framework’s alarm gate controls whether a model call is needed. Prompts exclude attack labels, clean targets, and case identifiers. Requests, responses, parser outcomes, tokens, latency, and graph paths are saved.

Rules: 41 improved, 7 unchanged, 0 harmed; correct building/channel in 41/48. One-pass Llama: 40 improved, 8 unchanged, 0 harmed; correct building/channel in 40/48. Full-graph Llama: 40 improved, 8 unchanged, 0 harmed; correct building/channel in 40/48. The two Llama modes harm 0 of their 8 clean-control executions. Across 165 actual model calls, 0 fail parsing or transport. Full-graph Llama requests 0 peer inspections and performs 1 replanning attempt. Llama closely matches the rules configuration on this subset, while supplying language-based diagnosis and tool selection within the shared workflow.

These configurations share CommunityGuard-AI’s tools and guard. The supplementary comparison describes their decision behavior and the additional inspection/retry options. Appendix A links the stages in a successful repair and a failed retry.

E. Execution Cost and Language-Agent Observations

One-pass Llama uses 82 calls, 52,607 input tokens, and 3,115 output tokens; 19,346 input tokens are served from cache (36.8%). Full-graph Llama uses 83 calls, 53,250 input tokens, and 3,128 output tokens; 39,370 input tokens are served from cache (73.9%). Counts are taken from the server’s prompt, response, and cached-prompt fields [16]; cached tokens are a subset of input tokens, not an additional workload. Median summed model-call time per record, including clean controls, is 11.1 s and 6.6 s, respectively. One-pass always precedes full-graph inference for each case, so caching and shared CPU execution confound a latency comparison. These are model-call measurements, not end-to-end control deadlines.

In a failed load case in cohort 2, Llama selects Building 1 although Building 4 is attacked. The guard rejects both its initial energy-balance proposal and its later peer-consensus proposal. LangGraph then retains the input. The three calls use 736/47, 589/29, and 648/23 input/output tokens. This shows that retry and rollback executed, but did not recover the case. In a separate clock response, Llama chooses the conditional model but cites the autoencoder’s improvement bound. A valid tool choice therefore does not guarantee an accurate explanation.

F. Attack Location, Execution, and Response Time

The full benchmark attacks each of Buildings 1–5 separately in each cohort: 192 attacked records per building. Figures 3–6

TABLE II
COMMUNITYGUARD-AI DECISION POLICIES ON MATCHED ATTACKS: BUILDING 3, COHORT 1, WEEK 1

Method	Tool	Edited / perturbed	Target RMSE	Tokens in / out	Forensic / planning (s)	Graph (s)
<i>Price (Fig. 3): $\tilde{C} = 0.6C + \epsilon$, noise SD 0.02; received RMSE = 0.120164 currency/kWh</i>						
Guarded rules	Peer	168 / 168	0.000000	0 / 0	–	NR
One-pass Llama	Cond.	168 / 168	0.000109	1,260 / 87	7.15 / 4.11	11.30
Full-graph Llama	Cond.	168 / 168	0.000109	1,258 / 92	2.93 / 5.12	8.08
<i>Load (Fig. 4): $\tilde{\ell} = 0.7\ell$; received RMSE = 0.2093 kWh</i>						
Guarded rules	Balance	162 / 168	0.0086	0 / 0	–	NR
One-pass Llama	Balance	162 / 168	0.0086	1,329 / 69	7.47 / 4.51	12.06
Full-graph Llama	Balance	162 / 168	0.0086	1,333 / 84	2.75 / 7.12	9.90
<i>PV (Fig. 5): $\tilde{p} = u_t p$, $u_t \sim \mathcal{U}(0.1, 0.5)$; received RMSE = 0.9403 kWh</i>						
Guarded rules	Balance	79 / 84	0.0046	0 / 0	–	NR
One-pass Llama	Balance	79 / 84	0.0046	1,323 / 72	8.36 / 5.75	14.16
Full-graph Llama	Balance	79 / 84	0.0046	1,323 / 71	2.12 / 4.61	6.77
<i>Clock (Fig. 6): 4-hour delay modulo 24; received RMSE = 4 h</i>						
Guarded rules	Peer	168 / 168	0	0 / 0	–	NR
One-pass Llama	Cond.	168 / 168	0	1,250 / 78	7.35 / 4.24	11.64
Full-graph Llama	Cond.	168 / 168	0	1,250 / 78	2.21 / 3.71	5.94

All three policies share the framework’s tools and guard. Attacks span 168 hourly samples; figures show the first 72. RMSE uses all 168 samples and the units stated above. Peer = peer consensus; Cond. = conditional model (peer-hour median for clock); Balance = energy balance. Tokens sum both Llama calls and include cached input tokens. – = no Llama calls; NR = rule-based wall-clock latency was not recorded. One-pass ran before full-graph, so caching confounds timing comparisons. Graph time excludes collection/training/scoring. Replay retains original live timings.

show four separate attacks on Building 3 of cohort 1, with Buildings 1, 2, 4, and 5 uncompromised. The examples use test week 1, trajectory indices 5376–5543 (zero-based). Each perturbation is applied throughout the 168-hour record; the first 72 hours are plotted. There is no uncorrupted pre-attack segment in these plots.

Table II compares three decision policies within CommunityGuard-AI on the same inputs. The four illustrated full-graph Llama cases select the correct building/channel and accept the first proposal, using two model calls each. For price, rules choose peer consensus and reach zero target error; Llama chooses the conditional predictor and leaves a small residual. Load, PV, and clock recovery match the rules configuration. In every case, the guard changes only accepted samples in the selected channel and preserves the other 139 coordinates. These examples show the complete path from attack evidence to a checked telemetry repair.

For Figures 3–6, latency is measured after the complete 168-hour input is available. Forensic and planning times are client-measured Llama calls. Total graph time spans graph construction, numerical evidence, both calls, verification, and trace/output recording; it excludes trajectory collection, training, and truth-based scoring. The remaining non-call work totals 16–35 ms per example; its individual stages were not separately timed. These CPU measurements use six inference threads. The hourly time axis describes simulated attack exposure; the seconds reported in Table II describe offline processing, not delay from attack onset. Replay preserves the recorded live timings and reports its own execution duration separately.

V. DISCUSSION AND LIMITATIONS

CommunityGuard-AI connects incident evidence, tool selection, selective verification, and recorded execution. Its rule-based configuration worsened 11 records, compared with 1089

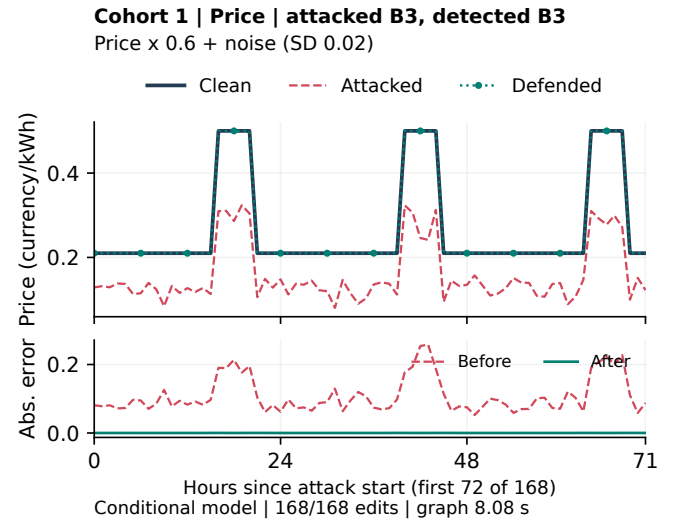


Fig. 3. Building 3 price attack and conditional-model repair. The ridge predictor uses peer observations and weather/calendar context. Table II reports the matched recovery, tokens, and latency.

for feature-aware autoencoder replacement. Tools and diagnosis also differ, so this comparison evaluates complete response designs. The workflow makes proposed changes inspectable and measures their effects against withheld clean data.

Llama supplied incident interpretations and tool choices, with recovery closely matching rules. Its saved explanations supported inspection but could be inaccurate. Verification and bounded fallback were shared framework capabilities. Superior adaptability, lower maintenance effort, and a recovery gain from Llama were not established. No peer inspection occurred, and the only retry failed, leaving their additional benefit unmeasured.

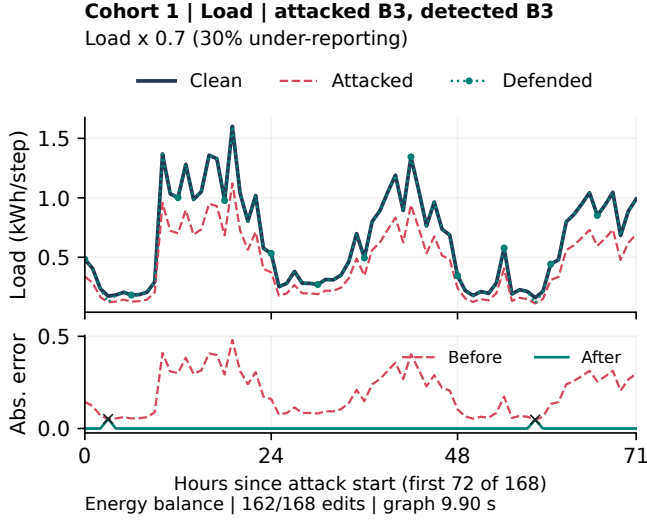


Fig. 4. Building 3 load attack. The energy-balance candidate $c = \max(0, n + \tilde{p})$ uses protected net import and received PV. Black crosses mark perturbed samples that the guard leaves unchanged.

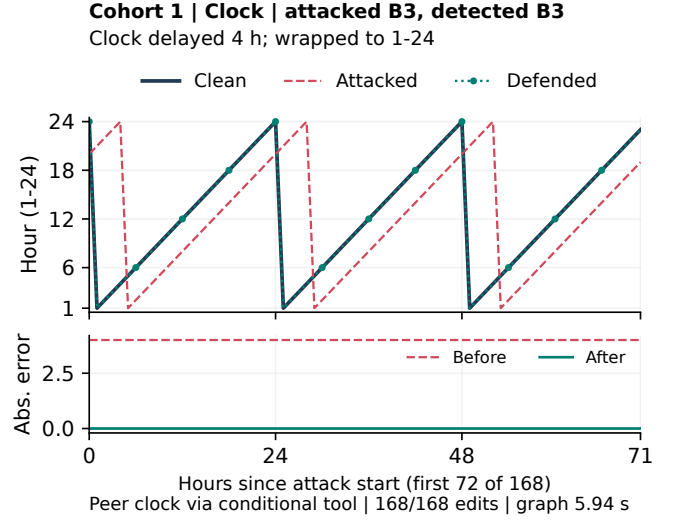


Fig. 6. Building 3 clock attack. The selected conditional tool returns the peer-hour median for this channel. The error panel uses circular distance on the 24-hour clock.

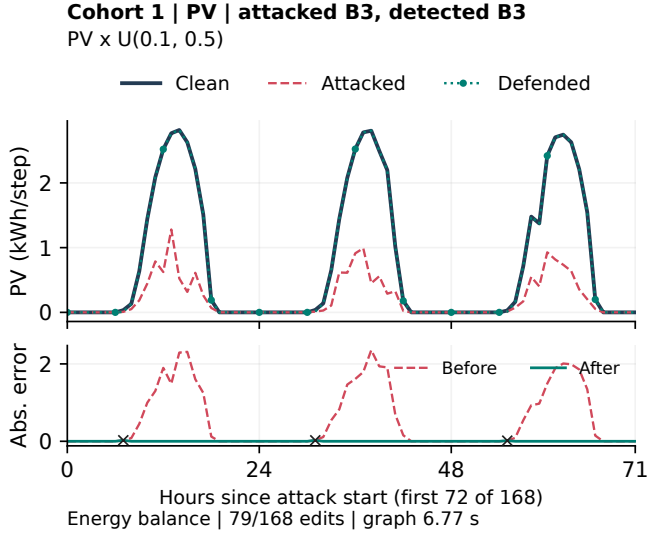


Fig. 5. Building 3 PV attack. The candidate $c = \max(0, \tilde{\ell} - n)$ uses received load and protected net import; transferred-PV evidence controls acceptance. Zero-PV samples are unaffected by the multiplicative attack.

The principal harmful failure confused load and PV within the correct building. Energy balance cannot identify which sensor is wrong; peer-based PV references help, but their errors can invalidate the recovery condition. The method assumes one compromised channel, honest peers, and a protected net meter. Seasonal change can exceed calibrated uncertainty, and the clean controls do not cover faults, maintenance, or legitimate sudden changes.

A separate 32-record stress experiment corrupts load and net import together while preserving their mutual balance. The guarded engine improves 0, leaves 32 unchanged, and harms 0. This common-mode failure exposes the dependence on an

independently protected measurement; these cases are excluded from the single-channel benchmark.

The two cohorts share an environment and calendar, and multiple attacks reuse the same weekly data. Counts therefore describe this experiment; they are not results from independent operating conditions. The Llama test uses one quantized model, seed, and CPU runtime. Offline repairs with zero control actions do not measure battery scheduling, energy cost, network constraints, or service continuity. Evaluating physical protection requires active-control or hardware tests and an energy balance that includes all relevant devices.

A. Implementation and Reproducibility

The companion package contains the code, recorded model responses, and tutorial; the original repository is cited in [17]. The tutorial restores fitted models, regenerates attacks, computes repairs, and scores them against withheld clean data. Tables and figures are generated from the current run and checked against its measurements. Replay requires matching prompts; tokens and latency retain their original live measurements. Retraining and fresh inference define new experiments. Separate scripted tests verify graph routing without claiming model performance.

VI. CONCLUSION

This work presented CommunityGuard-AI for diagnosing and repairing corrupted telemetry in energy communities. The framework integrated numerical evidence, repair tools, selective verification, and bounded LangGraph execution. A conditional proof stated when accepted changes reduced error. With rule-based decisions, the framework improved 1613 of 1920 attacked records, reduced mean normalized error by 79.2%, and worsened 11; it harmed 0 of 32 clean controls. Llama supplied language-based interpretation and tool selection;

its full-graph configuration improved 40 of 48 subset attacks, compared with 41 for rules. Independent evaluation documented a traceable offline repair workflow and its failure modes under explicit trust assumptions.

APPENDIX A

FRAMEWORK EVIDENCE AND REPAIR CONDITION

A. From Supplied Evidence to Executed Repair

Evidence supplied. For the load attack in Fig. 4, the forensic prompt asks for the best-supported building/channel. It reports that 98.214% of Building 3's load samples are inconsistent, above the 10% alarm threshold. The planning prompt lists four tools; only energy balance has eligible samples (162 hours). The complete received responses appear below; only whitespace was changed. Token counts include the full prompts and schemas. *Forensic response; 742 input / 51 output tokens.*

```
{
  "building": 3,
  "channel": "non_shiftable_load",
  "assessment": "corruption_suspected",
  "tool_request": "NONE",
  "justification": "Strong inconsistency with 98.214%
    inconsistent fraction, exceeding alarm fraction of
    0.1."
}
```

Planning response; 591 input / 33 output tokens.

```
{
  "operation": "ENERGY_BALANCE",
  "justification": "Eligible repair hours, 162 changed hours,
    and 0.1327 conditional improvement."
}
```

Executed result. LangGraph applies the selected tool; the guard edits 162 of 168 load samples and leaves all other channels unchanged. Load RMSE falls from 0.2093 to 0.0086 kWh. The response value 0.1327 is a mean conditional lower bound on absolute-error reduction over accepted samples, not the measured RMSE reduction.

Failure evidence. In cohort 2, Llama instead selects Building 1 during a Building 4 load attack. Both energy-balance and peer-consensus proposals fail verification; LangGraph stops after two attempts and retains every input value. Both full exchanges are in the tutorial. These cases document diagnosis, tool selection, verification, and fallback within the framework.

B. When an Accepted Change Reduces Error

For one sample, let x be the received value, c the proposed repair, x^* the unknown true value, and q the reference. Radius δ describes reference uncertainty; d is absolute distance or circular hour distance.

Proposition 1: If $d(x^*, q) \leq \delta$ and the guard in (10) accepts c , then $d(c, x^*) < d(x, x^*)$.

Proof: The triangle inequality bounds the repaired error above by $d(c, q) + \delta$. The reverse triangle inequality bounds the received error below by $d(x, q) - \delta$. The guard separates these bounds:

$$d(c, x^*) \leq d(c, q) + \delta < d(x, q) - \delta \leq d(x, x^*). \quad \blacksquare$$

Thus 2δ accounts for uncertainty in both comparisons. This condition supports the framework's guard under either decision

policy. It applies only when the reference-accuracy assumption holds; calibration alone does not ensure this on new data. Correct diagnosis and physical protection require separate evidence.

REFERENCES

- [1] K. Nweye, K. Kaspar, G. Buscemi, T. Fonseca, G. Pinto, D. Ghose, S. Duddukuru, P. Pratapa, H. Li, J. Mohammadi, L. L. Ferreira, T. Hong, M. Ouf, A. Capozzoli, and Z. Nagy, "CityLearn v2: Energy-flexible, resilient, occupant-centric, and carbon-aware management of grid-interactive communities," *J. Building Perform. Simul.*, vol. 18, no. 1, pp. 17–38, 2025, doi: 10.1080/19401493.2024.2418813.
- [2] M. Khalaf, A. Ayad, M. H. K. Tushar, M. Kassouf, and D. Kundur, "A survey on cyber-physical security of active distribution networks in smart grids," *IEEE Access*, vol. 12, pp. 29414–29444, 2024, doi: 10.1109/ACCESS.2024.3364362.
- [3] J. McCarthy, J. Marron, D. Faatz, D. Rebori-Carretero, J. Wiltberger, and N. Urlaub, "Cybersecurity for smart inverters: Guidelines for residential and light commercial solar energy systems," National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST Interagency/Internal Rep. 8498, Dec. 2024, doi: 10.6028/NIST.IR.8498.
- [4] Z. Zhang, S. Gong, A. D. Dimitrovski, and H. Li, "Time synchronization attack in smart grid: Impact and analysis," *IEEE Trans. Smart Grid*, vol. 4, no. 1, pp. 87–98, Mar. 2013, doi: 10.1109/TSG.2012.2227342.
- [5] Y. Liu, P. Ning, and M. K. Reiter, "False data injection attacks against state estimation in electric power grids," *ACM Trans. Inf. Syst. Secur.*, vol. 14, no. 1, Art. no. 13, pp. 1–33, 2011, doi: 10.1145/1952982.1952995.
- [6] M. J. Zideh, M. R. Khalghani, and S. K. Solanki, "An unsupervised adversarial autoencoder for cyber attack detection in power distribution grids," *Electr. Power Syst. Res.*, vol. 232, Art. no. 110407, Jul. 2024, doi: 10.1016/j.epsr.2024.110407.
- [7] S. Majumder, L. Dong, F. Doudi, Y. Cai, C. Tian, D. Kalathil, K. Ding, A. A. Thatte, N. Li, and L. Xie, "Exploring the capabilities and limitations of large language models in the electric energy sector," *Joule*, vol. 8, no. 6, pp. 1544–1549, Jun. 2024, doi: 10.1016/j.joule.2024.05.009.
- [8] G. Liu, Y. Tang, W. Guo, and J. Dang, "Redesigning power grid dispatching automation systems by integrating LLMs, KGs, and AI agents," *CSEE J. Power Energy Syst.*, vol. 11, no. 6, pp. 2610–2622, Nov. 2025, doi: 10.17775/CSEEJPES.2025.03480.
- [9] J. Ruan, G. Liang, H. Zhao, G. Liu, X. Sun, J. Qiu, Z. Xu, F. Wen, and Z. Y. Dong, "Applying large language models to power systems: Potential security threats," *IEEE Trans. Smart Grid*, vol. 15, no. 3, pp. 3333–3336, May 2024, doi: 10.1109/TSG.2024.3373256.
- [10] A. Srivastava, C.-C. Liu, A. Stefanov, S. Basumallik, M. M. Hussain, B. Somda, and V. S. Rajkumar, "Digital twins serving cybersecurity: More than a model," *IEEE Power Energy Mag.*, vol. 22, no. 1, pp. 61–71, Jan./Feb. 2024, doi: 10.1109/MPE.2023.3325196.
- [11] K. E. Kampourakis, V. Gkioulos, and S. Katsikas, "Systematic integration of digital twins and constrained LLMs for interpretable cyber-physical anomaly detection," arXiv:2604.03790, Apr. 2026, preprint, doi: 10.48550/arXiv.2604.03790.
- [12] Ollama, "Llama 3.2: 3B model," model distribution and configuration. [Online]. Available: <https://ollama.com/library/llama3.2:3b>
- [13] Ollama, "Structured outputs." [Online]. Available: <https://docs.ollama.com/capabilities/structured-outputs>
- [14] LangChain, "LangGraph overview." [Online]. Available: <https://docs.langchain.com/oss/python/langgraph/overview>
- [15] K. Nweye, Z. Nagy, S. Mohanty, D. Chakraborty, S. Sankaranarayanan, T. Hong, S. Dey, G. Henze, J. Drgona, F. Lin, W. Jiang, H. Zhang, Z. Yi, J. Zhang, C. Yang, M. Motoki, S. Khongnawang, M. Ibrahim, A. Zhumabekov, D. May, Z. Yang, X. Song, H. Zhang, X. Dong, S. Zheng, and J. Bian, "The CityLearn Challenge 2022: Overview, results, and lessons learned," in *Proc. NeurIPS 2022 Competitions Track*, ser. Proc. Mach. Learn. Res., vol. 220, pp. 85–103, 2022.
- [16] Ollama, "Generate a chat message," API reference. [Online]. Available: <https://docs.ollama.com/api/chat>
- [17] M. Bhadani, "CommunityGuard-AI: Source code and archived experiment outputs," GitHub repository, 2026. Accessed: Sep. 16, 2026. [Online]. Available: <https://github.com/maanathbhadani29/CommunityGuard-AI>